

Documentation

Prometheus + Grafana + Node Exporter

Version : 1.0

Date : Janvier 2026

Auteur : DESJARDINS Julien



Table des matières

Documentation	1
Prometheus + Grafana + Node Exporter.....	1
Table des matières.....	1
1. Introduction.....	5

1.1 Objectif	5
1.2 Cas d'usage	5
1.3 Versions	6
2. Prérequis	6
2.1 Système d'exploitation	6
2.2 Réseau	6
2.3 Outils nécessaires	7
3. Architecture	7
3.1 Schéma d'architecture	7
3.2 Flux de données	7
4. Installation Prometheus 3.9.1	8
4.1 Téléchargement	8
4.2 Création de l'utilisateur système	8
4.3 Création des répertoires	8
4.4 Attribution des permissions	8
4.5 Configuration initiale	8
4.6 Création du service systemd	9
4.7 Démarrage du service	10
4.8 Vérification	10
5. Installation Node Exporter	10
5.1 Installation sur le serveur Prometheus	10
5.1.1 Téléchargement	10
5.1.2 Création de l'utilisateur	10
5.1.3 Création du service	11
5.1.4 Démarrage	11
5.1.5 Vérification	11
5.2 Installation sur les serveurs clients Linux	11
5.2.1 Ouverture du pare-feu	11
5.2.2 Test depuis le serveur Prometheus	12
6. Installation Grafana 12.3	12
6.1 Ajout du dépôt	12
6.2 Installation	12

6.3 Démarrage.....	12
6.4 Vérification	13
6.5 Accès initial	13
7. Configuration Prometheus.....	13
7.1 Configuration multi-serveurs	13
7.2 Validation de la configuration.....	14
7.3 Rechargement de la configuration	15
7.4 Vérification des targets	15
8. Configuration Grafana	15
8.1 Ajout de Prometheus comme Data Source.....	15
8.1.1 Via l'interface web	15
8.1.2 Via API (optionnel)	15
8.2 Import de dashboards	16
8.2.1 Dashboard Node Exporter Full (ID: 1860)	16
8.2.2 Autres dashboards recommandés.....	16
8.3 Création d'un dashboard personnalisé	16
8.3.1 Créer un nouveau dashboard	16
8.3.2 Exemples de requêtes	16
9. Installation Windows Exporter.....	17
9.1 Téléchargement avec PowerShell.....	17
9.2 Installation	17
9.3 Vérification du service	18
9.4 Ouverture du pare-feu	18
9.5 Test.....	18
10. Configuration des Alertes.....	18
10.1 Création du répertoire d'alertes	18
10.2 Fichier d'alertes de base	18
10.3 Mise à jour de prometheus.yml	20
10.4 Rechargement	20
10.5 Vérification des alertes	20
11. Installation Alertmanager.....	21
11.1 Téléchargement	21

11.2 Création de l'utilisateur.....	21
11.3 Création des répertoires	21
11.4 Configuration	21
11.5 Création du service	22
11.6 Démarrage.....	23
11.7 Configuration dans Prometheus	23
11.8 Vérification	23
12. Découverte Automatique	23
12.1 Configuration File-based Service Discovery.....	23
12.1.1 Modifier prometheus.yml.....	24
12.1.2 Créer le répertoire	24
12.1.3 Script de découverte automatique.....	24
12.1.4 Rendre exécutable	25
12.1.5 Automatisation avec systemd timer.....	26
13. Maintenance et Dépannage	27
13.1 Commandes de vérification quotidiennes	27
13.1.1 Vérifier le statut des services.....	27
13.1.2 Vérifier les targets	27
13.1.3 Vérifier les alertes actives	27
13.2 Consultation des logs	27
13.3 Rechargement de configuration	28
13.4 Problèmes courants	28
13.4.1 Target DOWN dans Prometheus	28
13.4.2 Grafana affiche "No Data"	28
13.4.3 Erreur "User node_exporter not found".....	29
13.4.4 Prometheus consomme trop de disque	29
13.5 Sauvegarde.....	30
13.5.1 Sauvegarde de la configuration	30
13.5.2 Sauvegarde des données Prometheus	30
13.5.3 Sauvegarde Grafana	30
13.6 Mise à jour	31
13.6.1 Mise à jour Prometheus	31

13.6.2 Mise à jour Grafana.....	31
14. Annexes.....	32
14.1 Ports utilisés	32
14.2 Emplacements des fichiers	32
14.3 Requêtes PromQL utiles	32
CPU.....	32
Mémoire	33
Disque.....	33
Réseau	33
Système	34
14.4 Configuration Slack pour Alertmanager.....	34
14.5 Références	34
14.6 Checklist de déploiement	35

1. Introduction

1.1 Objectif

Cette documentation décrit l'installation et la configuration complète d'une stack de supervision basée sur :

- **Prometheus 3.9.1** : collecte et stockage des métriques
- **Grafana 12.3** : visualisation et dashboards
- **Node Exporter** : export des métriques système Linux
- **Windows Exporter** : export des métriques système Windows
- **Alertmanager** : gestion des alertes (optionnel)

1.2 Cas d'usage

Cette stack permet de :

- Superviser plusieurs serveurs Linux et Windows

- Visualiser en temps réel les métriques système (CPU, RAM, disque, réseau)
- Créer des alertes sur des seuils critiques
- Conserver l'historique des métriques
- Générer des rapports de performance

1.3 Versions

Composant	Version	Port par défaut
Prometheus	3.9.1	9090
Grafana	12.3.0	3000
Node Exporter	1.8.2	9100
Windows Exporter	0.27.2	9182
Alertmanager	0.27.0	9093

2. Prérequis

2.1 Système d'exploitation

Serveur de supervision (Prometheus + Grafana) :

- Debian 11/12 ou Ubuntu 20.04/22.04/24.04
- 2 CPU minimum
- 4 GB RAM minimum
- 50 GB disque minimum

Serveurs à superviser :

- Linux : Debian, Ubuntu, CentOS, RHEL
- Windows : Windows Server 2016+, Windows 10/11

2.2 Réseau

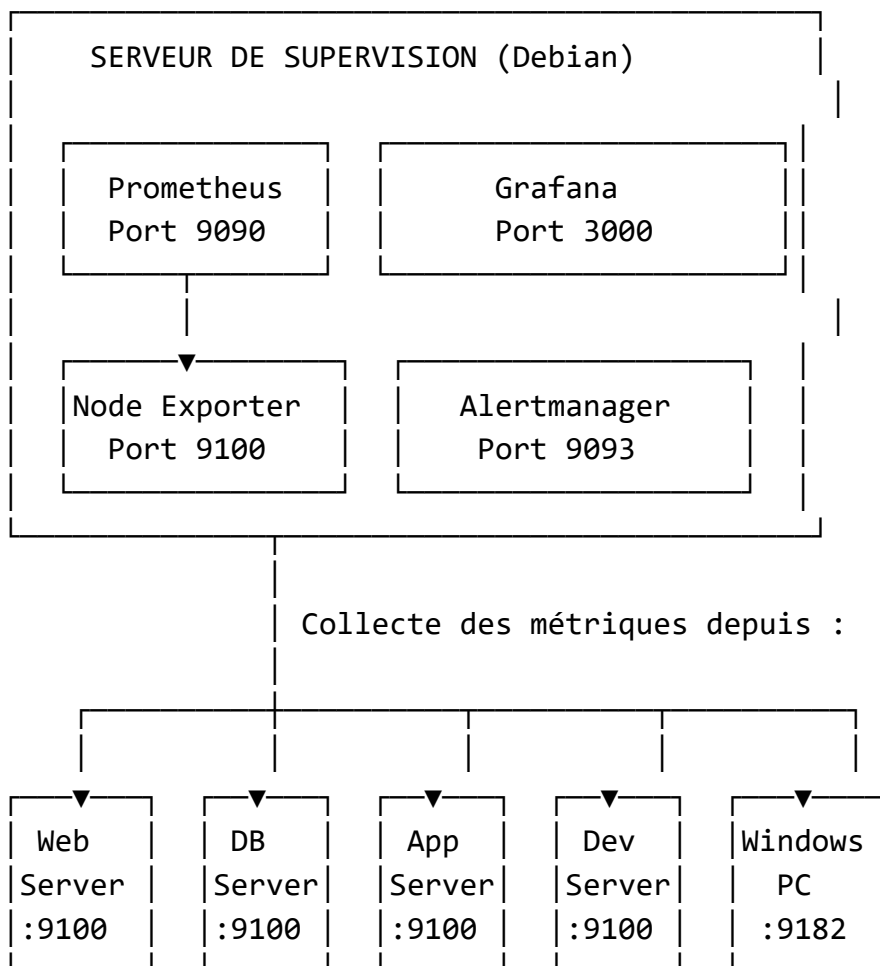
- Connectivité réseau entre le serveur Prometheus et les machines à superviser
- Ports à ouvrir :
 - Serveur Prometheus : 9090 (Prometheus), 3000 (Grafana), 9093 (Alertmanager)
 - Machines Linux : 9100 (Node Exporter)
 - Machines Windows : 9182 (Windows Exporter)

2.3 Outils nécessaires

```
sudo apt-get update  
sudo apt-get install -y wget curl tar gnupg apt-transport-https  
software-properties-common
```

3. Architecture

3.1 Schéma d'architecture



3.2 Flux de données

1. **Node Exporter** expose les métriques système sur le port 9100
2. **Prometheus** collecte (scrape) ces métriques toutes les 15 secondes

3. **Prometheus** stocke les métriques dans sa base de données TSDB
4. **Grafana** interroge Prometheus pour afficher les dashboards
5. **Alertmanager** reçoit les alertes de Prometheus et envoie les notifications

4. Installation Prometheus 3.9.1

4.1 Téléchargement

```
cd /tmp
wget
https://github.com/prometheus/prometheus/releases/download/v3.9.1/prometheus-3.9.1.linux-amd64.tar.gz
tar xvfz prometheus-3.9.1.linux-amd64.tar.gz
sudo mv prometheus-3.9.1.linux-amd64 /opt/prometheus
```

4.2 Création de l'utilisateur système

```
sudo useradd --no-create-home --shell /bin/false prometheus
```

4.3 Création des répertoires

```
sudo mkdir -p /etc/prometheus /var/lib/prometheus
sudo cp /opt/prometheus/prometheus.yml /etc/prometheus/
sudo cp -r /opt/prometheus/consoles /etc/prometheus/
sudo cp -r /opt/prometheus/console_libraries /etc/prometheus/
```

4.4 Attribution des permissions

```
sudo chown -R prometheus:prometheus /opt/prometheus /etc/prometheus
/var/lib/prometheus
```

4.5 Configuration initiale

Éditer le fichier de configuration :

```
sudo nano /etc/prometheus/prometheus.yml
```


Contenu de base :

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    monitor: 'production-monitor'

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
```

4.6 Création du service systemd

```
sudo tee /etc/systemd/system/prometheus.service > /dev/null <<'EOF'
[Unit]
Description=Prometheus Monitoring System
Documentation=https://prometheus.io/docs/introduction/overview/
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/opt/prometheus/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus/ \
  --web.console.templates=/etc/prometheus/consoles \
  --web.console.libraries=/etc/prometheus/console_libraries

ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF
```

4.7 Démarrage du service

```
sudo systemctl daemon-reload
sudo systemctl start prometheus
sudo systemctl enable prometheus
sudo systemctl status prometheus
```

4.8 Vérification

```
# Test de santé
curl http://localhost:9090/-/healthy

# Test de l'API
curl http://localhost:9090/api/v1/status/config

# Interface web
echo "Accès : http://localhost:9090"
```

5. Installation Node Exporter

5.1 Installation sur le serveur Prometheus

5.1.1 Téléchargement

```
cd /tmp
wget
https://github.com/prometheus/node\_exporter/releases/download/v1.8.2/node\_exporter-1.8.2.linux-amd64.tar.gz
tar xvfz node_exporter-1.8.2.linux-amd64.tar.gz
sudo mv node_exporter-1.8.2.linux-amd64 /opt/node_exporter
```

5.1.2 Création de l'utilisateur

```
sudo useradd --no-create-home --shell /bin/false node_exporter
sudo chown -R node_exporter:node_exporter /opt/node_exporter
```

5.1.3 Création du service

```
sudo tee /etc/systemd/system/node_exporter.service > /dev/null
<<'EOF'
[Unit]
Description=Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/opt/node_exporter/node_exporter

[Install]
WantedBy=multi-user.target
EOF
```

5.1.4 Démarrage

```
sudo systemctl daemon-reload
sudo systemctl start node_exporter
sudo systemctl enable node_exporter
sudo systemctl status node_exporter
```

5.1.5 Vérification

```
curl http://localhost:9100/metrics | head -20
```

5.2 Installation sur les serveurs clients Linux

Répéter les étapes 5.1.1 à 5.1.5 sur chaque serveur Linux à superviser.

5.2.1 Ouverture du pare-feu

```
sudo ufw allow 9100/tcp
```

Ou pour firewalld :

```
sudo firewall-cmd --permanent --add-port=9100/tcp
sudo firewall-cmd --reload
```

5.2.2 Test depuis le serveur Prometheus

```
# Remplacer IP_SERVEUR_CLIENT par l'IP réelle
curl http://IP\_SERVEUR\_CLIENT:9100/metrics
```

6. Installation Grafana 12.3

6.1 Ajout du dépôt

```
sudo apt-get update
sudo apt-get install -y gnupg apt-transport-https software-
properties-common wget
sudo mkdir -p /etc/apt/keyrings/
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo
tee /etc/apt/keyrings/grafana.gpg > /dev/null
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg]
https://apt.grafana.com stable main" | sudo tee
/etc/apt/sources.list.d/grafana.list
```

6.2 Installation

```
sudo apt-get update
sudo apt-get install -y grafana=12.3.0
```

6.3 Démarrage

```
sudo systemctl daemon-reload
sudo systemctl start grafana-server
sudo systemctl enable grafana-server
sudo systemctl status grafana-server
```

6.4 Vérification

curl <http://localhost:3000/api/health>

6.5 Accès initial

URL : http://IP_SERVEUR:3000

Identifiants par défaut :

- Username : admin
- Password : admin

Note : Grafana demandera de changer le mot de passe à la première connexion.

7. Configuration Prometheus

7.1 Configuration multi-serveurs

Éditer le fichier de configuration :

```
sudo nano /etc/prometheus/prometheus.yml
```

Configuration complète :

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    monitor: 'production-monitor'

# Fichiers de règles d'alertes
rule_files:
  - '/etc/prometheus/alerts/*.yaml'

# Configuration des targets
scrape_configs:
  # Prometheus lui-même
  - job_name: 'prometheus'
```

```

static_configs:
  - targets: ['localhost:9090']
    labels:
      alias: 'serveur-prometheus'

# Serveur Prometheus (métriques système)
- job_name: 'serveur-prometheus'
  static_configs:
    - targets: ['localhost:9100']
      labels:
        alias: 'serveur-supervision'
        type: 'infrastructure'

# Serveurs Linux
- job_name: 'serveurs-linux'
  static_configs:
    - targets:
        - '192.168.1.101:9100'
        - '192.168.1.102:9100'
        - '192.168.1.103:9100'
      labels:
        environment: 'production'
        type: 'linux'

# Serveurs Windows
- job_name: 'serveurs-windows'
  static_configs:
    - targets:
        - '192.168.1.200:9182'
        - '192.168.1.201:9182'
      labels:
        environment: 'production'
        type: 'windows'

```

Note : Remplacer les adresses IP par celles de vos serveurs.

7.2 Validation de la configuration

```
/opt/prometheus/promtool check config /etc/prometheus/prometheus.yml
```

7.3 Rechargement de la configuration

```
sudo systemctl reload prometheus
```

7.4 Vérification des targets

Interface web : http://IP_SERVEUR:9090/targets

Ou en ligne de commande :

```
curl http://localhost:9090/api/v1/targets | jq  
'data.activeTargets[] | {job: .labels.job, instance:  
.labels.instance, health: .health}'
```

8. Configuration Grafana

8.1 Ajout de Prometheus comme Data Source

8.1.1 Via l'interface web

1. Se connecter à Grafana : http://IP_SERVEUR:3000
2. Aller dans  **Menu** → **Connections** → **Data sources**
3. Cliquer sur **Add data source**
4. Sélectionner **Prometheus**
5. Configurer :
 - a. **Name** : Prometheus
 - b. **URL** : <http://localhost:9090>
 - c. **Access** : Server (default)
6. Cliquer sur **Save & Test**

Le message suivant doit apparaître :  **"Successfully queried the Prometheus API."**

8.1.2 Via API (optionnel)

```
curl -X POST http://admin:admin@localhost:3000/api/datasources \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "Prometheus",
```

```
"type": "prometheus",  
"url": "http://localhost:9090",  
"access": "proxy",  
"isDefault": true  
'}
```

8.2 Import de dashboards

8.2.1 Dashboard Node Exporter Full (ID: 1860)

1. Dans Grafana : ☰ → **Dashboards** → **New** → **Import**
2. Dans le champ "**Grafana.com dashboard URL or ID**", taper : 1860
3. Cliquer sur **Load**
4. Sélectionner la data source **Prometheus**
5. Cliquer sur **Import**

8.2.2 Autres dashboards recommandés

Dashboard	ID	Description
Node Exporter Full	1860	Dashboard complet pour Linux
Node Exporter Server Metrics	11074	Vue multi-serveurs
Windows Node	10467	Dashboard pour Windows
Prometheus Stats	3662	Statistiques Prometheus

8.3 Création d'un dashboard personnalisé

8.3.1 Créer un nouveau dashboard

1. ☰ → **Dashboards** → **New Dashboard**
2. **Add visualization**
3. Sélectionner **Prometheus**

8.3.2 Exemples de requêtes

CPU Usage :

```
100 - (avg by(instance)  
(rate(node_cpu_seconds_total{mode="idle"}[5m]))) * 100)
```


Memory Usage :

```
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes))  
* 100
```

Disk Usage :

```
100 - ((node_filesystem_avail_bytes{mountpoint="/",fstype!="rootfs"}  
* 100) /  
node_filesystem_size_bytes{mountpoint="/",fstype!="rootfs"})
```

Network Traffic (RX) :

```
rate(node_network_receive_bytes_total[5m])
```

Network Traffic (TX) :

```
rate(node_network_transmit_bytes_total[5m])
```

9. Installation Windows Exporter

9.1 Téléchargement avec PowerShell

Ouvrir **CMD en tant qu'administrateur** sur la machine Windows :

```
powershell -Command "Invoke-WebRequest -Uri  
'https://github.com/prometheus-  
community/windows\_exporter/releases/download/v0.27.2/windows\_exporte  
r-0.27.2-amd64.msi' -OutFile '%TEMP%\windows_exporter.msi'"
```

9.2 Installation

```
msiexec /i %TEMP%\windows_exporter.msi  
ENABLED_COLLECTORS="cpu,cs,logical_disk,net,os,system,memory,process  
" /qn
```

9.3 Vérification du service

```
sc query windows_exporter
```

Le service doit afficher : STATE : 4 RUNNING

9.4 Ouverture du pare-feu

```
netsh advfirewall firewall add rule name="Windows Exporter" dir=in  
action=allow protocol=TCP localport=9182
```

9.5 Test

Depuis la machine Windows :

```
start http://localhost:9182/metrics
```

Depuis le serveur Prometheus :

```
curl http://IP\_WINDOWS:9182/metrics
```

10. Configuration des Alertes

10.1 Création du répertoire d'alertes

```
sudo mkdir -p /etc/prometheus/alerts
```

10.2 Fichier d'alertes de base

```
sudo nano /etc/prometheus/alerts/basic-alerts.yml
```

Contenu :

```
groups:  
  - name: basic_alerts  
    interval: 30s
```

```

rules:
  # Instance down
  - alert: InstanceDown
    expr: up == 0
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "Instance {{ $labels.instance }} est DOWN"
      description: "{{ $labels.instance }} de job {{ $labels.job }} est inaccessible depuis plus de 5 minutes."

  # CPU élevé
  - alert: HighCPUUsage
    expr: 100 - (avg by(instance)
(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
    for: 10m
    labels:
      severity: warning
    annotations:
      summary: "CPU élevé sur {{ $labels.instance }}"
      description: "Le CPU est à {{ $value }}% sur {{ $labels.instance }}"

  # Mémoire critique
  - alert: HighMemoryUsage
    expr: (1 - (node_memory_MemAvailable_bytes /
node_memory_MemTotal_bytes)) * 100 > 90
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "Mémoire critique sur {{ $labels.instance }}"
      description: "Utilisation mémoire : {{ $value }}%"

  # Disque plein (warning)
  - alert: DiskSpaceLow
    expr: (1 - (node_filesystem_avail_bytes{mountpoint="/" } /
node_filesystem_size_bytes{mountpoint="/" }))) * 100 > 80
    for: 5m
    labels:
      severity: warning

```

```

    annotations:
      summary: "Espace disque faible sur {{ $labels.instance }}"
      description: "Espace disque utilisé : {{ $value }}"

# Disque plein (critical)
- alert: DiskSpaceCritical
  expr: (1 - (node_filesystem_avail_bytes{mountpoint="/" } /
node_filesystem_size_bytes{mountpoint="/" })) * 100 > 90
  for: 1m
  labels:
    severity: critical
  annotations:
    summary: "Espace disque CRITIQUE sur {{ $labels.instance
}}"
    description: "URGENT : Espace disque utilisé : {{ $value
}}% !"

```

10.3 Mise à jour de prometheus.yml

Ajouter la section `rule_files` dans `/etc/prometheus/prometheus.yml` :

```

rule_files:
  - '/etc/prometheus/alerts/*.yaml'

```

10.4 Rechargement

```
sudo systemctl reload prometheus
```

10.5 Vérification des alertes

Interface web : http://IP_SERVEUR:9090/alerts

Ou en ligne de commande :

```
curl http://localhost:9090/api/v1/rules
```

11. Installation Alertmanager

11.1 Téléchargement

```
cd /tmp
wget
https://github.com/prometheus/alertmanager/releases/download/v0.27.0/alertmanager-0.27.0.linux-amd64.tar.gz
tar xvfz alertmanager-0.27.0.linux-amd64.tar.gz
sudo mv alertmanager-0.27.0.linux-amd64 /opt/alertmanager
```

11.2 Création de l'utilisateur

```
sudo useradd --no-create-home --shell /bin/false alertmanager
```

11.3 Création des répertoires

```
sudo mkdir -p /etc/alertmanager /var/lib/alertmanager
sudo chown -R alertmanager:alertmanager /opt/alertmanager
/etc/alertmanager /var/lib/alertmanager
```

11.4 Configuration

```
sudo nano /etc/alertmanager/alertmanager.yml
```

Configuration de base (email) :

```
global:
  resolve_timeout: 5m
  smtp_smarthost: 'smtp.gmail.com:587'
  smtp_from: 'votre-email@gmail.com'
  smtp_auth_username: 'votre-email@gmail.com'
  smtp_auth_password: 'votre-mot-de-passe-app'
  smtp_require_tls: true

route:
  group_by: ['alertname', 'cluster', 'service']
  group_wait: 10s
```

```
group_interval: 10s
repeat_interval: 12h
receiver: 'email-notifications'
```

```
receivers:
```

```
- name: 'email-notifications'
```

```
  email_configs:
```

```
    - to: 'admin@example.com'
```

```
      headers:
```

```
        Subject: '🚨 ALERTE - {{ .GroupLabels.alertname }}'
```

```
inhibit_rules:
```

```
- source_match:
```

```
  severity: 'critical'
```

```
target_match:
```

```
  severity: 'warning'
```

```
equal: ['alertname', 'instance']
```

11.5 Création du service

```
sudo tee /etc/systemd/system/alertmanager.service > /dev/null
```

```
<<'EOF'
```

```
[Unit]
```

```
Description=Alertmanager
```

```
Wants=network-online.target
```

```
After=network-online.target
```

```
[Service]
```

```
User=alertmanager
```

```
Group=alertmanager
```

```
Type=simple
```

```
ExecStart=/opt/alertmanager/alertmanager \
```

```
--config.file=/etc/alertmanager/alertmanager.yml \
```

```
--storage.path=/var/lib/alertmanager/
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

11.6 Démarrage

```
sudo systemctl daemon-reload
sudo systemctl start alertmanager
sudo systemctl enable alertmanager
sudo systemctl status alertmanager
```

11.7 Configuration dans Prometheus

Ajouter dans `/etc/prometheus/prometheus.yml` :

```
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - 'localhost:9093'
```

Puis recharger :

```
sudo systemctl reload prometheus
```

11.8 Vérification

```
curl http://localhost:9093/-/healthy
```

Interface web : http://IP_SERVEUR:9093

12. Découverte Automatique

12.1 Configuration File-based Service Discovery

Cette méthode permet de découvrir automatiquement les machines via un fichier JSON.

12.1.1 Modifier prometheus.yml

```
scrape_configs:
  - job_name: 'auto-discovery'
    file_sd_configs:
      - files:
          - '/etc/prometheus/targets/*.json'
        refresh_interval: 1m
```

12.1.2 Créer le répertoire

```
sudo mkdir -p /etc/prometheus/targets
```

12.1.3 Script de découverte automatique

```
sudo nano /usr/local/bin/prometheus-discover.sh
```

Contenu :

```
#!/bin/bash

SUBNET="192.168.1"
OUTPUT="/etc/prometheus/targets/auto-discovered.json"
TEMP="/tmp/prometheus-targets.json"

echo '[' > "$TEMP"
first=true

for i in {1..254}; do
    ip="$SUBNET.$i"

    # Test Node Exporter (Linux)
    if timeout 1 curl -sf "http://\$ip:9100/metrics" >/dev/null 2>&1;
then
    [ "$first" = false ] && echo ',' >> "$TEMP"
    cat >> "$TEMP" <<JSON
{
  "targets": ["$ip:9100"],
  "labels": {
    "type": "linux",
```



```

        "discovered": "$(date +%Y-%m-%d_%H:%M)"
    }
}
JSON
    first=false
    echo "✅ Linux node: $ip"
fi

# Test Windows Exporter
if timeout 1 curl -sf "http://\$ip:9182/metrics" >/dev/null 2>&1;
then
    [ "$first" = false ] && echo ',' >> "$TEMP"
    cat >> "$TEMP" <<JSON
    {
        "targets": ["$ip:9182"],
        "labels": {
            "type": "windows",
            "discovered": "$(date +%Y-%m-%d_%H:%M)"
        }
    }
JSON
    first=false
    echo "✅ Windows node: $ip"
fi
done

echo ']' >> "$TEMP"
mv "$TEMP" "$OUTPUT"

if /opt/prometheus/promtool check config
/etc/prometheus/prometheus.yml; then
    systemctl reload prometheus
    echo "✅ Découverte terminée et Prometheus rechargé"
else
    echo "❌ Erreur de configuration"
fi

```

12.1.4 Rendre exécutable

```
sudo chmod +x /usr/local/bin/prometheus-discover.sh
```

12.1.5 Automatisation avec systemd timer

Créer le service :

```
sudo tee /etc/systemd/system/prometheus-discover.service > /dev/null
<<'EOF'
[Unit]
Description=Prometheus Auto Discovery

[Service]
Type=oneshot
ExecStart=/usr/local/bin/prometheus-discover.sh
EOF
```

Créer le timer :

```
sudo tee /etc/systemd/system/prometheus-discover.timer > /dev/null
<<'EOF'
[Unit]
Description=Run Prometheus Auto Discovery every 5 minutes

[Timer]
OnBootSec=1min
OnUnitActiveSec=5min

[Install]
WantedBy=timers.target
EOF
```

Activer :

```
sudo systemctl daemon-reload
sudo systemctl enable prometheus-discover.timer
sudo systemctl start prometheus-discover.timer
```

Vérifier :

```
sudo systemctl status prometheus-discover.timer
```

13. Maintenance et Dépannage

13.1 Commandes de vérification quotidiennes

13.1.1 Vérifier le statut des services

```
sudo systemctl status prometheus
sudo systemctl status grafana-server
sudo systemctl status node_exporter
sudo systemctl status alertmanager
```

13.1.2 Vérifier les targets

```
curl http://localhost:9090/api/v1/targets | jq
'.data.activeTargets[] | {job: .labels.job, instance:
.labels.instance, health: .health}'
```

13.1.3 Vérifier les alertes actives

```
curl http://localhost:9090/api/v1/alerts | jq '.data.alerts[] |
select(.state == "firing")'
```

13.2 Consultation des logs

```
# Prometheus
sudo journalctl -u prometheus -f

# Grafana
sudo journalctl -u grafana-server -f

# Node Exporter
sudo journalctl -u node_exporter -f

# Alertmanager
sudo journalctl -u alertmanager -f
```

13.3 Rechargement de configuration

```
# Prometheus (sans redémarrage)
sudo systemctl reload prometheus

# Grafana (nécessite redémarrage)
sudo systemctl restart grafana-server

# Alertmanager
sudo systemctl reload alertmanager
```

13.4 Problèmes courants

13.4.1 Target DOWN dans Prometheus

Symptôme : Une machine apparaît en rouge dans /targets

Diagnostic :

```
# Depuis le serveur Prometheus, tester la connexion
curl http://IP\_MACHINE:9100/metrics

# Vérifier le pare-feu
sudo ufw status
```

Solutions :

```
# Sur la machine cliente
sudo systemctl status node_exporter
sudo systemctl restart node_exporter
sudo ufw allow 9100/tcp
```

13.4.2 Grafana affiche "No Data"

Diagnostic :

```
# Vérifier la data source
curl http://localhost:3000/api/datasources

# Tester Prometheus
```

curl <http://localhost:9090/api/v1/query?query=up>

Solutions :

1. Dans Grafana :  → **Connections** → **Data sources** → **Prometheus** → **Save & Test**
2. Vérifier que Prometheus collecte : http://IP_SERVEUR:9090/targets

13.4.3 Erreur "User node_exporter not found"

Solution :

```
# Recréer l'utilisateur
sudo useradd --no-create-home --shell /bin/false node_exporter
sudo chown -R node_exporter:node_exporter /opt/node_exporter
sudo systemctl restart node_exporter
```

13.4.4 Prometheus consomme trop de disque

Diagnostic :

```
du -sh /var/lib/prometheus
```

Solution - Configurer la rétention :

Modifier /etc/systemd/system/prometheus.service :

```
ExecStart=/opt/prometheus/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus/ \
  --storage.tsdb.retention.time=30d \
  --storage.tsdb.retention.size=50GB
```

Puis recharger :

```
sudo systemctl daemon-reload
sudo systemctl restart prometheus
```

13.5 Sauvegarde

13.5.1 Sauvegarde de la configuration

```
# Créer une archive
sudo tar -czf prometheus-backup-$(date +%Y%m%d).tar.gz \
  /etc/prometheus/ \
  /etc/alertmanager/ \
  /etc/grafana/

# Copier vers un emplacement sûr
sudo cp prometheus-backup-*.tar.gz /backup/
```

13.5.2 Sauvegarde des données Prometheus

```
# Arrêter Prometheus
sudo systemctl stop prometheus

# Sauvegarder les données
sudo tar -czf prometheus-data-$(date +%Y%m%d).tar.gz
/var/lib/prometheus/

# Redémarrer
sudo systemctl start prometheus
```

13.5.3 Sauvegarde Grafana

```
# Exporter les dashboards
curl -H "Authorization: Bearer VOTRE_API_KEY" \
  http://localhost:3000/api/search?type=dash-db | \
  jq -r '.[ ] | .uid' | \
  xargs -I {} curl -H "Authorization: Bearer VOTRE_API_KEY" \
  http://localhost:3000/api/dashboards/uid/{} > dashboard-{}.json
```

13.6 Mise à jour

13.6.1 Mise à jour Prometheus

```
# Télécharger la nouvelle version
cd /tmp
wget
https://github.com/prometheus/prometheus/releases/download/vX.Y.Z/pr  
ometheus-X.Y.Z.linux-amd64.tar.gz

# Arrêter le service
sudo systemctl stop prometheus

# Sauvegarder l'ancienne version
sudo mv /opt/prometheus /opt/prometheus.old

# Installer la nouvelle version
tar xvfz prometheus-X.Y.Z.linux-amd64.tar.gz
sudo mv prometheus-X.Y.Z.linux-amd64 /opt/prometheus
sudo chown -R prometheus:prometheus /opt/prometheus

# Copier la configuration
sudo cp /etc/prometheus/prometheus.yml /opt/prometheus/

# Redémarrer
sudo systemctl start prometheus
sudo systemctl status prometheus
```

13.6.2 Mise à jour Grafana

```
sudo apt-get update
sudo apt-get install grafana=X.Y.Z
sudo systemctl restart grafana-server
```

14. Annexes

14.1 Ports utilisés

Service	Port	Protocole	Description
Prometheus	9090	TCP	Interface web et API
Grafana	3000	TCP	Interface web
Node Exporter	9100	TCP	Métriques système Linux
Windows Exporter	9182	TCP	Métriques système Windows
Alertmanager	9093	TCP	Interface web et API

14.2 Emplacements des fichiers

Type	Emplacement
Binares Prometheus	/opt/prometheus/
Configuration Prometheus	/etc/prometheus/prometheus.yml
Règles d'alertes	/etc/prometheus/alerts/*.yaml
Données Prometheus	/var/lib/prometheus/
Binaire Node Exporter	/opt/node_exporter/
Configuration Grafana	/etc/grafana/grafana.ini
Données Grafana	/var/lib/grafana/
Binaire Alertmanager	/opt/alertmanager/
Configuration Alertmanager	/etc/alertmanager/alertmanager.yml

14.3 Requêtes PromQL utiles

CPU

```
# CPU Usage par serveur
100 - (avg by(instance)
(rate(node_cpu_seconds_total{mode="idle"}[5m]))) * 100)
```

```
# CPU par core
100 - (avg by(instance, cpu)
(rate(node_cpu_seconds_total{mode="idle"}[5m]))) * 100)
```

```
# Load Average
node_load1
node_load5
```


node_load15

Mémoire

```
# Mémoire utilisée (%)
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes))
* 100
```

```
# Mémoire disponible (GB)
node_memory_MemAvailable_bytes / 1024 / 1024 / 1024
```

```
# Swap utilisé (%)
(1 - (node_memory_SwapFree_bytes / node_memory_SwapTotal_bytes)) *
100
```

Disque

```
# Espace disque utilisé (%)
100 - ((node_filesystem_avail_bytes{mountpoint="/" } * 100) /
node_filesystem_size_bytes{mountpoint="/" })
```

```
# Espace disponible (GB)
node_filesystem_avail_bytes{mountpoint="/" } / 1024 / 1024 / 1024
```

```
# IOPS lecture
rate(node_disk_reads_completed_total[5m])
```

```
# IOPS écriture
rate(node_disk_writes_completed_total[5m])
```

Réseau

```
# Trafic entrant (MB/s)
rate(node_network_receive_bytes_total[5m]) / 1024 / 1024
```

```
# Trafic sortant (MB/s)
rate(node_network_transmit_bytes_total[5m]) / 1024 / 1024
```

```
# Erreurs réseau
```

```
rate(node_network_receive_errs_total[5m])
rate(node_network_transmit_errs_total[5m])
```

Système

```
# Uptime (jours)
(time() - node_boot_time_seconds) / 86400
```

```
# Nombre de processus
node_procs_running
```

```
# Température CPU (si disponible)
node_hwmon_temp_celsius
```

14.4 Configuration Slack pour Alertmanager

```
receivers:
  - name: 'slack-notifications'
    slack_configs:
      - api_url:
'https://hooks.slack.com/services/VOTRE/WEBHOOK/URL'
        channel: '#alertes'
        title: '🚨 {{ .GroupLabels.alertname }}'
        text: |
          {{ range .Alerts }}
            *Alerte:* {{ .Labels.alertname }}
            *Sévérité:* {{ .Labels.severity }}
            *Instance:* {{ .Labels.instance }}
            *Description:* {{ .Annotations.description }}
          {{ end }}
        send_resolved: true
```

14.5 Références

- **Documentation Prometheus** : <https://prometheus.io/docs/>
- **Documentation Grafana** : <https://grafana.com/docs/>
- **Dashboards Grafana** : <https://grafana.com/grafana/dashboards/>
- **Node Exporter** : https://github.com/prometheus/node_exporter

- **Windows Exporter** : https://github.com/prometheus-community/windows_exporter
- **Alertmanager** : <https://prometheus.io/docs/alerting/latest/alertmanager/>

14.6 Checklist de déploiement

Serveur de supervision :

- ☐ Prometheus installé et démarré
- ☐ Grafana installé et démarré
- ☐ Node Exporter installé (pour surveiller le serveur lui-même)
- ☐ Alertmanager installé (optionnel)
- ☐ Ports ouverts : 9090, 3000, 9093
- ☐ Configuration prometheus.yml validée
- ☐ Data source Prometheus ajoutée dans Grafana
- ☐ Dashboards importés

Serveurs clients Linux :

- ☐ Node Exporter installé et démarré
- ☐ Port 9100 ouvert dans le pare-feu
- ☐ Accessible depuis le serveur Prometheus
- ☐ Ajouté dans prometheus.yml
- ☐ Target visible en vert dans /targets

Postes clients Windows :

- ☐ Windows Exporter installé
- ☐ Port 9182 ouvert dans le pare-feu
- ☐ Accessible depuis le serveur Prometheus
- ☐ Ajouté dans prometheus.yml
- ☐ Target visible en vert dans /targets

Configuration des alertes :

- ☐ Fichiers d'alertes créés
- ☐ Alertes visibles dans /alerts
- ☐ Alertmanager configuré (si utilisé)
- ☐ Notifications testées

Tests fonctionnels :

- ☐ Toutes les targets UP dans Prometheus
- ☐ Dashboards Grafana affichent des données
- ☐ Alertes fonctionnelles
- ☐ Rétention des données configurée
- ☐ Sauvegarde configurée

FIN DE LA DOCUMENTATION

Version : 1.0

Date de dernière mise à jour : Janvier 2026